

Bloki anonimowe

PostgreSQL

PL/pgSQL

- PL/pgSQL to język proceduralny dla systemu baz danych PostgreSQL. Celem projektowym PL/pgSQL było stworzenie języka proceduralnego, który
 - może być używany do tworzenia funkcji, procedur i wyzwalaczy,
 - dodaje struktury kontrolne do języka SQL,
 - może wykonywać złożone obliczenia,
 - dziedziczy wszystkie typy, funkcje, procedury i operatory zdefiniowane przez użytkownika,
 - może być zdefiniowany jako zaufany przez serwer,
 - jest łatwy w użyciu.

Zalety korzystania z PL/pgSQL

- SQL to język, którego PostgreSQL i większość innych relacyjnych baz danych używa jako języka zapytań. Jest przenośny i łatwy do nauczenia. Jednak każda instrukcja SQL musi być wykonywana indywidualnie przez serwer bazy danych.
- Oznacza to, że aplikacja kliencka musi wysłać każde zapytanie do serwera bazy danych, poczekać na jego przetworzenie, odebrać i przetworzyć wyniki, wykonać pewne obliczenia, a następnie wysłać kolejne zapytania do serwera.
- Wszystko to wiąże się z komunikacją międzyprocesową, a także obciążeniem sieci, jeśli klient znajduje się na innej maszynie niż serwer bazy danych.

Zalety korzystania z PL/pgSQL

- Dzięki PL/pgSQL można zgrupować blok obliczeń i serię zapytań wewnątrz serwera bazy danych, mając w ten sposób moc języka proceduralnego i łatwość użycia SQL, ale ze znaczną oszczędnością kosztów komunikacji klient/serwer
 - Dodatkowe ruchy w obie strony między klientem a serwerem są wyeliminowane
 - Wyniki pośrednie, których klient nie potrzebuje, nie muszą być przesyłane między serwerem a klientem.
 - Można uniknąć wielu rund analizowania zapytań
- Może to spowodować znaczny wzrost wydajności w porównaniu do aplikacji, która nie korzysta z funkcji przechowywanych.
- Ponadto w PL/pgSQL można używać wszystkich typów danych, operatorów i funkcji SQL.

Struktura PL/pgSQL

- PL/pgSQL jest językiem o strukturze blokowej. Cały tekst ciała funkcji musi być blokiem. Blok jest zdefiniowany jako:

```
[ <<label>> ]  
[ DECLARE  
    declarations ]  
BEGIN  
    statements  
END [ label ];
```

- Każda deklaracja i każda instrukcja w bloku jest zakończona średnikiem.
- Blok, który pojawia się wewnątrz innego bloku, musi mieć średnik po END, jak pokazano powyżej; jednak końcowy END, który kończy ciało funkcji, nie wymaga średnika.

Struktura PL/pgSQL

```
[ <<label>> ]  
[ DECLARE  
    declarations ]  
BEGIN  
    statements  
END [ label ];
```

- **Label** - jest potrzebny tylko wtedy, gdy chcesz zidentyfikować blok do użycia w instrukcji EXIT lub zakwalifikować nazwy zmiennych zadeklarowanych w bloku. Jeśli etykieta jest podana po END, musi być zgodna z etykietą na początku bloku.
- **Wielkość liter nie ma znaczenia dla wszystkich słów kluczowych.** Identyfikatory są niejawnie konwertowane na małe litery, chyba że są podwójnie cytowane, tak jak w zwykłych poleceniach SQL.
- Komentarze działają tak samo w kodzie PL/pgSQL jak w zwykłym SQL.
 - Podwójny myślnik (--) rozpoczyna komentarz, który rozciąga się do końca linii.
 - Znak /* rozpoczyna komentarz blokowy, który rozciąga się do pasującego wystąpienia znaku */. Komentarze blokowe zagnieżdżają się.
- Ważne jest, aby nie mylić użycia BEGIN/END do grupowania instrukcji w PL/pgSQL z podobnie nazwanymi poleceniami SQL do kontroli transakcji. **Polecenia BEGIN/END w PL/pgSQL służą wyłącznie do grupowania; nie rozpoczynają ani nie kończą transakcji**

Deklaracje

- Wszystkie zmienne używane w bloku muszą być zadeklarowane w sekcji deklaracji bloku.
 - Jedynym wyjątkiem jest zmienna pętli FOR iterująca po zakresie wartości całkowitych, gdyż jest automatycznie deklarowana jako zmienna całkowita i podobnie zmienna pętli FOR iterująca po wyniku kursora jest automatycznie deklarowana jako zmienna rekordowa.

```
user_id integer;  
quantity numeric(5);  
url varchar;  
myrow tablename%ROWTYPE;  
myfield tablename.columnname%TYPE;  
arow RECORD;
```

```
name [ CONSTANT ] type [ COLLATE collation_name ] [ NOT NULL ] [ { DEFAULT | := | = } expression ];
```

Deklaracje

```
name [ CONSTANT ] type [ COLLATE collation_name ] [ NOT NULL ] [ { DEFAULT | := | = } expression ] ;
```

- Klauzula DEFAULT, jeśli jest podana, określa początkową wartość przypisaną do zmiennej po wprowadzeniu bloku.
- Jeśli klauzula DEFAULT nie zostanie podana, zmienna zostanie zainicjowana do wartości SQL null.
 - Wartość domyślna zmiennej jest obliczana i przypisywana do zmiennej za każdym razem, gdy wprowadzany jest blok (nie tylko raz na wywołanie funkcji). Tak więc, na przykład, przypisanie now() do zmiennej typu timestamp powoduje, że zmienna ma czas bieżącego wywołania funkcji, a nie czas, w którym funkcja została wstępnie skompilowana.
- Opcja CONSTANT zapobiega przypisaniu zmiennej po inicjalizacji, dzięki czemu jej wartość pozostanie stała przez cały czas trwania bloku.
- Opcja COLLATE można zdefiniować, w jaki sposób tekst ma być sortowany i porównywany, co jest szczególnie ważne w przypadku języków o różnych zasadach sortowania (np. uwzględniających akcenty czy wielkość liter).
- Jeśli określono NOT NULL, przypisanie wartości null spowoduje błąd w czasie wykonywania. Wszystkie zmienne zadeklarowane jako NOT NULL muszą mieć określoną wartość domyślną inną niż null.
- Równość (=) może być używana zamiast zgodnego z PL/SQL :=

Deklaracje przykłady

```
quantity integer DEFAULT 32;  
url varchar := 'http://mysite.com';  
transaction_time CONSTANT timestamp with time zone := now();
```

```
DECLARE  
  x integer := 1;  
  y integer := x + 1;
```

Alias

```
newname ALIAS FOR oldname;
```

- można zadeklarować alias dla dowolnej zmiennej, nie tylko parametrów funkcji.
- Głównym praktycznym zastosowaniem tego jest przypisanie innej nazwy dla zmiennych o z góry określonych nazwach, takich jak NEW lub OLD w wyzwalaczu.

```
DECLARE  
  prior ALIAS FOR old;  
  updated ALIAS FOR new;
```

Kopiowanie typów

```
variable%TYPE
```

- %TYPE określa typ danych zmiennej lub kolumny tabeli.
- Można go użyć do zadeklarowania zmiennych, które będą przechowywać wartości bazy danych.
- Załóżmy na przykład, że w tabeli users znajduje się kolumna o nazwie user_id. Aby zadeklarować zmienną o tym samym typie danych co users.user_id należy napisać:

```
user_id users.user_id%TYPE;
```

- Używając %TYPE nie musisz znać typu danych struktury, do której się odwołujesz,
- jeśli typ danych elementu, do którego się odwołujesz, zmieni się w przyszłości (na przykład: zmienisz typ user_id z całkowitego na rzeczywisty), nie będziesz musiał zmieniać definicji funkcji.

Przypisanie

- Przypisanie wartości do zmiennej PL/pgSQL jest zapisywane jako:

```
variable { := | = } expression;
```

- Jeśli typ danych wyniku wyrażenia nie pasuje do typu danych zmiennej, wartość zostanie wymuszona

Wykonywanie polecenia z wynikiem w jednym wierszu

- Wynik polecenia SQL dającego pojedynczy wiersz (ewentualnie z wieloma kolumnami) można przypisać do zmiennej rekordowej, zmiennej typu wierszowego lub listy zmiennych skalarnych(target).
- Odbywa się to poprzez napisanie podstawowego polecenia SQL i dodanie klauzuli INTO.

```
SELECT select_expressions INTO [STRICT] target FROM ...;  
INSERT ... RETURNING expressions INTO [STRICT] target;  
UPDATE ... RETURNING expressions INTO [STRICT] target;  
DELETE ... RETURNING expressions INTO [STRICT] target;
```

- Zmienne PL/pgSQL zostaną podstawione do reszty polecenia (czyli wszystkiego poza klauzulą INTO), tak jak opisano powyżej, a plan jest buforowany w ten sam sposób.
- Działa to dla SELECT, INSERT/UPDATE/DELETE z RETURNING i niektórych poleceń narzędziowych, które zwracają zestawy wierszy, takie jak EXPLAIN.

Wykonywanie
polecenia z
wynikiem w
jednym
wierszu

```
DO $$  
DECLARE  
v_liczba_pracownikow NUMERIC;  
BEGIN  
    SELECT count(*) INTO v_liczba_pracownikow  
    FROM employees;  
    RAISE NOTICE ' %', v_liczba_pracownikow;  
END $$;
```

Tip

- Interpretacja SELECT z INTO jest zupełnie inna niż zwykłe polecenie SELECT INTO PostgreSQL, w którym celem INTO jest nowo utworzona tabela.
- Jeśli chcesz utworzyć tabelę z wyniku SELECT wewnątrz funkcji PL/pgSQL, użyj składni CREATE TABLE AS SELECT.

Instrukcje warunkowe

- Instrukcje IF-THEN są najprostszą formą IF. Instrukcje pomiędzy THEN i END IF zostaną wykonane, jeśli warunek jest prawdziwy. W przeciwnym razie są one pomijane.

```
IF boolean-expression THEN  
    statements  
END IF;
```

```
IF boolean-expression THEN  
    statements  
ELSE  
    statements  
END IF;
```

```
IF boolean-expression THEN  
    statements  
[ ELSIF boolean-expression THEN  
    statements  
[ ELSIF boolean-expression THEN  
    statements  
    ...  
]  
]  
[ ELSE  
    statements ]  
END IF;
```


Instrukcje warunkowe

- Prosta forma CASE zapewnia warunkowe wykonanie na podstawie równości operandów.
- Szukane wyrażenie jest oceniane (raz) i kolejno porównywane z każdym wyrażeniem w klauzulach WHEN.
- Jeśli zostanie znalezione dopasowanie, wykonywane są odpowiednie instrukcje, a następnie sterowanie przechodzi do następnej instrukcji po END CASE.

```
CASE search-expression
  WHEN expression [, expression [ ... ]] THEN
    statements
  [ WHEN expression [, expression [ ... ]] THEN
    statements
    ... ]
  [ ELSE
    statements ]
END CASE;
```

Pętle

- LOOP definiuje bezwarunkową pętlę, która jest powtarzana w nieskończoność aż do zakończenia przez instrukcję EXIT lub RETURN.
- Opcjonalna etykieta(label) może być używana przez instrukcje EXIT i CONTINUE wewnątrz zagnieżdżonych pętli w celu określenia, do której pętli odnoszą się te instrukcje.

```
[ <<label>> ]  
LOOP  
    statements  
END LOOP [ label ];
```

Pętle

- EXIT może być używany ze wszystkimi typami pętli; nie jest ograniczony do użycia z pętlami bezwarunkowymi.
- W przypadku użycia z blokiem BEGIN, EXIT przekazuje kontrolę do następnej instrukcji po zakończeniu bloku. Zauważ, że etykieta musi być użyta w tym celu; nieoznaczony EXIT nigdy nie jest uważany za pasujący do bloku BEGIN

```
LOOP
  -- some computations
  IF count > 0 THEN
    EXIT; -- exit loop
  END IF;
END LOOP;
```

```
LOOP
  -- some computations
  EXIT WHEN count > 0; -- same result as previous example
END LOOP;
```

```
<<ablock>>
BEGIN
  -- some computations
  IF stocks > 100000 THEN
    EXIT ablock; -- causes exit from the BEGIN block
  END IF;
  -- computations here will be skipped when stocks > 100000
END;
```

Pętle - continue

- Polecenie **CONTINUE** w pętlach PostgreSQL (PL/pgSQL) umożliwia pominięcie pozostałych instrukcji bieżącej iteracji pętli i natychmiastowe przejście do następnej iteracji.
- **Do czego służy CONTINUE**
 - **Pomija resztę kodu** w bloku pętli, gdy znajdzie określony warunek.
 - Służy do efektywnego sterowania przebiegiem pętli, np. jeśli dla pewnych danych nie chcesz wykonywać dalszych operacji w tej iteracji, lecz od razu przejść do kolejnego kroku.
 - Działa we wszystkich rodzajach pętli: LOOP, FOR, WHILE

```
DO $$
DECLARE
    cnt int = 0;
BEGIN
    LOOP
        cnt := cnt + 1;
        EXIT WHEN cnt > 10;
        CONTINUE WHEN mod(cnt,2) = 1; -- pomija liczby nieparzyste
        RAISE NOTICE '%', cnt; -- wypisuje tylko liczby parzyste
    END LOOP;
END;
$$;
```

Pętle - while

```
[ <<label>> ]  
WHILE boolean-expression LOOP  
    statements  
END LOOP [ label ];
```

- Instrukcja WHILE powtarza sekwencję instrukcji tak długo, jak długo wyrażenie logiczne ma wartość true. Wyrażenie jest sprawdzane tuż **PRZED** każdym wejściem do ciała pętli.

```
WHILE amount_owed > 0 AND gift_certificate_balance > 0 LOOP  
    -- some computations here  
END LOOP;  
  
WHILE NOT done LOOP  
    -- some computations here  
END LOOP;
```

Pętle - for

```
[ <<label>> ]  
FOR name IN [ REVERSE ] expression .. expression [ BY expression ] LOOP  
    statements  
END LOOP [ label ];
```

- Ta forma FOR tworzy pętlę, która iteruje po zakresie wartości całkowitych. Dwa wyrażenia określające dolną i górną granicę zakresu są obliczane raz przy wejściu do pętli.
- Jeśli klauzula BY nie jest określona, krok iteracji wynosi 1, w przeciwnym razie jest to wartość określona w klauzuli BY, która ponownie jest obliczana raz przy wejściu do pętli.
- Jeśli określono REVERSE, wartość kroku jest odejmowana, a nie dodawana, po każdej iteracji.

```
FOR i IN 1..10 LOOP  
    -- i will take on the values 1,2,3,4,5,6,7,8,9,10 within the loop  
END LOOP;
```

```
FOR i IN REVERSE 10..1 LOOP  
    -- i will take on the values 10,9,8,7,6,5,4,3,2,1 within the loop  
END LOOP;
```

```
FOR i IN REVERSE 10..1 BY 2 LOOP  
    -- i will take on the values 10,8,6,4,2 within the loop  
END LOOP;
```

Wyjątki

- Domyślnie każdy błąd występujący w funkcji PL/pgSQL przerywa wykonywanie funkcji i otaczającej ją transakcji. Błędy można wychwycić i odzyskać po ich wystąpieniu za pomocą bloku BEGIN z klauzulą EXCEPTION. Składnia jest rozszerzeniem normalnej składni bloku BEGIN:

```
[ <<label>> ]  
[ DECLARE  
    declarations ]  
BEGIN  
    statements  
EXCEPTION  
    WHEN condition [ OR condition ... ] THEN  
        handler_statements  
    [ WHEN condition [ OR condition ... ] THEN  
        handler_statements  
    ... ]  
END;
```